

Tutorium 5

Listen & Listenkomprehension

Grundlagen der Programmierung

15. Juni 2026

Listen in Haskell

Listenkomprehension

Praxis

Was ist eine Liste?

Eine Liste ist ein algebraischer Datentyp, der rekursiv definiert ist:

$$\text{data } [a] = [] \mid a : [a]$$

- ▶ `[]` ist die **leere Liste** (Basisfall).
- ▶ `a : [a]` ist eine Liste mit einem **Kopf (Head)** und einem **Rest (Tail)**.

Beispiel: Listenaufbau

```
1  -- Listenliteral als Syntax Sugar
2  [1, 2, 3, 4] == 1 : 2 : 3 : 4 : []
3
4  -- Pattern Matching auf Listen
5  head' :: [a] -> a
6  head' (x:xs) = x
```

Zusatz: Strings sind einfach Listen von Characters: `"Hallo" == ['H', 'a', 'l', 'l', 'o']`.

Operationen auf Listen

Da Listen rekursiv sind, werden Operationen meist über **Pattern Matching** und Rekursion gelöst.

Element anfügen (Cons)

```
1 cons :: [a] -> a -> [a]
2 cons [] el      = [el]
3 cons (x:xs) el  = x : cons xs el
```

n-tes Element finden

```
1 nth :: Int -> [a] -> a
2 nth 0 (x:_)  = x
3 nth _ []     = error "Out of
4               bounds"
5 nth n (_:xs) = nth (n-1) xs
```

Listenkomprehension (List Comprehension)

Kompakte Schreibweise, um neue Listen aus alten zu erzeugen (ähnlich der Mengenlehre in der Mathematik).

Struktur: [Ausdruck | Generator, Filter]

Beispiele

```
1  -- Alle Quadratzahlen von 1 bis 10
2  squares = [x^2 | x <- [1..10]]
3
4  -- Nur gerade Quadratzahlen (mit Filter)
5  evenSquares = [x^2 | x <- [1..10], even x]
6
7  -- Vielfache von 3
8  multiplesOfThree = [x | x <- [1..30], x 'mod' 3 == 0]
```

Live Demo



`Listen.hs & Listenkomprehension.hs`

Gemeinsame Programmierung von Listenoperationen.

Aufgaben:

1. Ein Element in einer Liste suchen (`findFirst`, `firstGreaterThan`)
2. Elemente an einem Index einfügen (`insertAt`)
3. Gefilterte Listen (z. B. `removeTwos`)